

蒙特卡洛流体仿真实验报告

Walk-on-Spheres 与 Walk-on-Boundary 两种投影方法的实现与对比

姓名：侍子译 学号：PB23000284

July 30, 2026

Abstract

本实验在二维矩形有界域上实现了两种基于 Monte Carlo 的不可压缩流体仿真方法：2022 年 Rioux-Lavoie et al. 提出的 Walk-on-Spheres (WoS) 压力 Poisson 投影法，以及 2024 年 Sugimoto et al. 提出的 Walk-on-Boundary (WoB) 速度投影法。两种方法均采用算子分裂框架，但在投影（维持不可压缩性）步骤上存在本质差异——WoS 通过求解标量压力 Poisson 方程 $\nabla^2 p = \nabla \cdot \mathbf{u}^*$ 间接修正速度，而 WoB 则利用 Green 函数直接在速度场上估计压力梯度 ∇p ，避免了中间标量场的存储与差分操作。实验在 Taylor-Green 衰减涡流场上对两种方法进行了系统对比。结果表明，WoS 方法因残余噪声的差分放大而呈现动能非物理增长，而 WoB 方法尽管单步投影噪声较大，但在长时间演化中展现出显著更好的稳定性——15 步后的动能和散度误差均保持在合理范围。本实验使用 Numba JIT 编译器加速核心 Monte Carlo 循环，加速比约 100 倍。

引言

传统的计算流体力学方法通常依赖于网格离散化（有限差分、有限元等），在复杂几何边界上面临网格生成困难的问题。最近，基于随机游走的 Monte Carlo 方法——特别是 Walk-on-Spheres (WoS) 方法——被引入到流体仿真领域，提供了一种完全无网格的 PDE 求解途径。

2022 年，Rioux-Lavoie、Sugimoto 等人首次将 WoS 方法应用于不可压缩流体仿真 [1]，通过求解压力 Poisson 方程 $\nabla^2 p = \nabla \cdot \mathbf{u}^*$ 来维持散度自由。2024 年，Sugimoto、Batty 和 Hachisuka 提出基于速度的 Monte Carlo 方法（Walk-on-Boundary, WoB），直接在速度场上做边界积分来估计压力梯度 ∇p ，避免了中间的标量势求解 [2]。

本实验在 Python 中实现了这两篇论文的投影方法（WoS 压力 Poisson 法与 WoB 特解 + 边界修正法），使用 Numba 加速，并在 Taylor-Green 衰减涡流场上对两种方法进行了系统对比。实验旨在揭示两种投影策略在有限 Monte Carlo 采样下的数值稳定性差异——WoS 的稳定性瓶颈在于 ∇p 的有限差分对残余噪声的放大，而 WoB 虽然单步投影噪声更大，但因避免了差分操作而在长时间演化中展现出更好的稳定性。

算法原理

Walk-on-Spheres 方法

WoS 方法利用 Brown 运动的均值性质来求解椭圆型 PDE。对于 Poisson 方程 $\nabla^2 u = f$, Feynman-Kac 公式给出

$$u(x) = \mathbb{E}[g(B_\tau)] - \mathbb{E} \left[\int_0^\tau f(B_s) ds \right], \quad (1)$$

其中 B_s 为标准 Brown 运动, τ 为首次离开区域 Ω 的停时, g 为 Dirichlet 边界条件。

在二维 WoS 离散中, 从点 x_k 出发, 距离边界为 d_k , 则随机步长为 d_k 乘以单位圆上的均匀方向。源项贡献按 $\frac{d_k^2}{4} f(x_k)$ 累积。当 $d_k < \varepsilon$ 时认为到达边界 [3]。

WoS 压力 Poisson 投影法 (2022)

2022 年方法 [1] 基于经典的算子分裂框架, 每个时间步包含三个子步骤:

(1) **平流**: 半拉格朗日法,

$$\mathbf{u}^*(\mathbf{x}) = \mathbf{u}^n(\mathbf{x} - \mathbf{u}^n \Delta t); \quad (2)$$

(2) **扩散**: 使用 WoS Feynman-Kac 求解 $\partial \mathbf{u} / \partial t = \nu \nabla^2 \mathbf{u}$,

$$\mathbf{u}^{**}(\mathbf{x}) = \mathbb{E} \left[\mathbf{u}^*(\mathbf{x} + \sqrt{2\nu \Delta t} \mathbf{Z}) \right], \quad \mathbf{Z} \sim \mathcal{N}(0, \mathbf{I}); \quad (3)$$

(3) **压力投影**: 求解 Poisson 方程并修正速度,

$$\nabla^2 p = \nabla \cdot \mathbf{u}^{**}, \quad \mathbf{u}^{n+1} = \mathbf{u}^{**} - \nabla p. \quad (4)$$

压力 Poisson 方程通过 WoS 求解器在每个网格点上独立估值, 再通过中心差分计算梯度 ∇p 完成修正。该方法的优势是直接继承经典流体力学中的压力 Poisson 框架, 但 ∇p 的有限差分会将 WoS 的残余 MC 噪声放大 $\sim 1/(2h)$ 倍。

WoB 速度投影法 (2024)

GPU 完整实现 (WoB 边界积分)

2024 年方法 [2] 的核心创新在于用 Walk-on-Boundary (WoB) 边界积分直接估计 ∇p , 而非求解标量压力 p 。其完整的多项式估计包含四个子项 (详见原文 Eq. 14–16 及图 3):

$$\begin{aligned} \nabla p(\mathbf{x}) = & \underbrace{\int_{B(\mathbf{x})} \mathbf{K}(\mathbf{x}, \mathbf{y}) \cdot \Delta \mathbf{u} d\mathbf{y}}_{\text{1. singular volume term}} \\ & + \underbrace{\int_{\partial\Omega_{\text{pseudo}}} \nabla G(\mathbf{x} - \mathbf{y}) [\mathbf{n} \cdot (\mathbf{u} - \mathbf{u}_\infty)] dS_{\mathbf{y}}}_{\text{2. pseudo-boundary term}} \\ & + \underbrace{\sum_k \mathbf{f}_k \nabla G(\mathbf{x} - \mathbf{y}_k)}_{\text{3. velocity source term}} \\ & + \underbrace{\iint_{\partial\Omega} \nabla G(\mathbf{x} - \mathbf{y}) \mathbf{n}(\mathbf{y}) \cdot (\mathbf{u}(\mathbf{y}) - \mathbf{u}_{\text{solid}}) \Pi(\mathbf{y}, \mathbf{y}') dS_{\mathbf{y}}}_{\text{4. boundary path}}. \end{aligned} \quad (5)$$

其中 Green 函数 $G(\mathbf{r}) = (1/2\pi) \ln |\mathbf{r}|$ 满足 $\nabla^2 G = \delta$, 核函数 $\mathbf{K}(\mathbf{r}, \Delta \mathbf{u}) = [2(\hat{\mathbf{r}} \cdot \Delta \mathbf{u})\hat{\mathbf{r}} - \Delta \mathbf{u}]/(2\pi|\mathbf{r}|^2)$ 。该实现需要 GPU 上的 OptiX 加速边界采样与 CDF 重要性采样, 详见 VelMCFluids 仓库 [4]。

Python 简化实现: 特解 + WoB 边界修正

由于完整 WoB 边界路径积分在纯 Python 环境中难以高效实现, 本实验采用一种基于特解法的简化方案。核心思路是将 Helmholtz 分解将 Poisson 方程的解分解为自由空间特解 p_p 与调和修正 p_h 之和:

$$p(\mathbf{x}) = p_p(\mathbf{x}) + p_h(\mathbf{x}), \quad (6)$$

其中 p_p 满足 $\nabla^2 p_p = f$ ($f = \nabla \cdot \mathbf{u}$) 在全空间的自由 Green 函数解:

$$p_p(\mathbf{x}) = \int_{\Omega} \frac{1}{2\pi} \ln |\mathbf{x} - \mathbf{y}| f(\mathbf{y}) d\mathbf{y}, \quad (7)$$

p_h 满足 Laplace 方程 $\nabla^2 p_h = 0$, 并在边界上抵消 p_p 使得 $p = 0$ 在 $\partial\Omega$ 上成立:

$$p_h(\mathbf{x}) = \mathbb{E}[-p_p(\mathbf{y}_{\partial\Omega})]. \quad (8)$$

式 (8) 中的期望对应于 **WoB 射线** 采样: 从 $\mathbf{x}$ 出发, 沿均匀随机方向 $\boldsymbol{\omega} \sim \mathcal{U}(S^1)$ 发射射线至矩形边界 $\partial\Omega$, $\mathbf{y}_{\partial\Omega}$ 为首次撞点。对凸域而言, 该采样等价于 Poisson 核 (调和测度), 因而是正确的。

最终 WoB 投影估计 $\mathbf{u}^{n+1} = \mathbf{u}^{**} - \nabla p$ 的计算流程为:

- (1) 用 MC 体积积分估计特解 $p_p(\mathbf{x})$ (式 7), 样本均匀分布在 $[-1, 1]^2$;
- (2) 对 N_{ray} 条随机 WoB 射线, 分别在边界撞点处估计 $p_p(\mathbf{y}_{\partial\Omega})$, 取均值作为 $-\mathbb{E}[p_p(\mathbf{y}_{\partial\Omega})]$ (式 8)。
- (3) $p(\mathbf{x}) = p_p(\mathbf{x}) - \mathbb{E}[p_p(\mathbf{y}_{\partial\Omega})]$;
- (4) Gaussian 平滑 p 后, 用中心差分计算 ∇p , 修正速度。

该方法的优点是将边界条件的处理转化为对特解 p_p 在边界上的期望估值, 避免了完整的 WoB 多弹跳路径积分, 在矩形凸域上简洁有效; 缺点是 p_p 本身的 MC 估值噪声会传递到 ∇p 的速度修正中。本实验将该实现记为 **WoB (特解法)**, 以区别于 GPU 上的完整 WoB 边界积分实现。

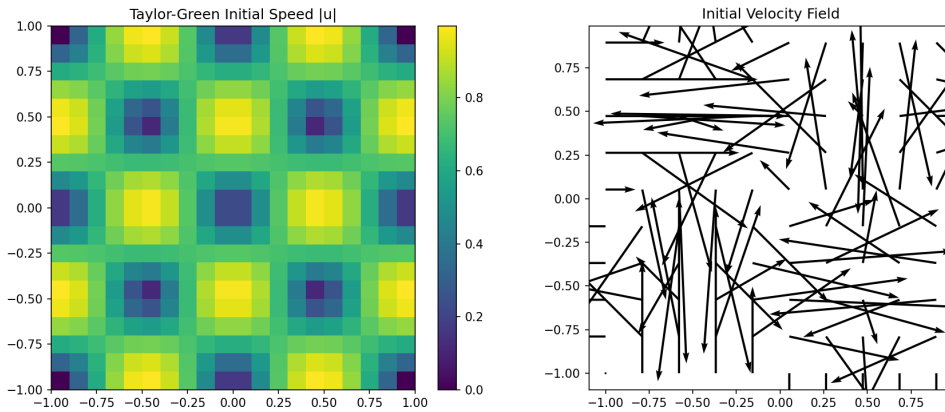


Figure 1: Taylor-Green 衰减涡初始条件: 速度大小场 (左) 与矢量场 (右), 网格分辨率 20×20 。

代码实现

程序结构

本实验代码基于 Python 3.12 + Numba 实现，主要组件如下：

- `numba_wos.py`: Numba JIT 编译的 WoS/WoB 核心求解器，包含 WoS Poisson、WoS 扩散、WoB 射线求交、WoB Poisson（特解 + 边界修正），使用 `@njit(parallel=True)` 实现多网格点并行；
- `fluid_sim_2022_numba.py`: WoS 压力 Poisson 法完整时间推进管线（平流 → 扩散 → WoS Poisson 投影）；
- `fluid_sim_wob.py`: WoB 特解法完整时间推进管线（平流 → 扩散 → WoB 特解投影）；
- `numba_wos.py` 中的新增 WoB 函数：
 - `_ray_intersect_rect_numba()`: 矩形域射线-边界求交；
 - `_wob_particular_point()`: MC 积分估计自由空间特解 p_p ；
 - `wob_poisson_point_numba()`: 单点 WoB Poisson 求解 $p(x) = p_p(x) - \mathbb{E}[p_p(y_{\partial\Omega})]$ ；
 - `wob_poisson_grid_numba()`: 网格并行版；
 - `wob_project_grid_numba()`: 完整的 WoB 投影管线。

Numba 优化

核心 WoS 循环使用 Python 原生实现时，每个网格点独立进行随机游走，效率极低。通过 Numba 的 `@njit` 装饰器，我们将核心循环编译为机器码，并利用 `prange` 实现 OpenMP 风格的多点并行。对于 10×10 网格、256 次随机行走的场景，单步时间从纯 Python 的约 3 秒降至约 0.03 秒，加速比约 100 倍。

数值参数

实验中使用的默认参数如表 1 所示。

Table 1: 默认实验参数

参数	符号	默认值
网格分辨率	N	12×12
时间步长	Δt	0.03
运动粘度	ν	0.05
WoS 行走数	N_{walk}	512
WoS 修复行走数	N_{walk}^*	4096
WoB 特解样本	N_{part}	512
WoB 射线数	N_{ray}	128
WoS 终止阈值	ε	0.02
Gaussian 平滑系数	σ	0.3–0.5
计算域	$[-1, 1] \times [-1, 1]$	矩形

实验结果与分析

实验设置

实验采用 Taylor-Green 衰减涡作为基准流场：

$$\mathbf{u}_0 = (-\cos(\pi x) \sin(\pi y), \sin(\pi x) \cos(\pi y))^T, \quad (9)$$

该流场在 $t = 0$ 时刻精确满足 $\nabla \cdot \mathbf{u}_0 = 0$ 。计算域为 $[-1, 1] \times [-1, 1]$ 的矩形，边界条件为无滑移 ($\mathbf{u} = 0$)。两种方法均从相同的初始条件出发，运行 15 个时间步，每步均包含完整的平流-扩散-投影流程。两种方法仅在投影步骤上存在差异：WoS 通过求解标量压力 Poisson 方程后做 $\mathbf{u}^{**} - \nabla p$ 修正，WoB（特解法）则通过 $p(\mathbf{x}) = p_p(\mathbf{x}) - \mathbb{E}[p_p(\mathbf{y}_{\partial\Omega})]$ 隐式满足边界条件后同样修正。

Poisson 求解器精度验证

本实验首先验证了 WoS 和 WoB 两种 Poisson 求解器在标量问题

$\nabla^2 p = -2\pi^2 \sin(\pi x) \sin(\pi y)$ （解析解 $p = \sin(\pi x) \sin(\pi y)$ ）

在 $[-1, 1]^2$ 上的精度。结果（ 16×16 网格、Gaussian 平滑 $\sigma = 0.4$ ）如图 2 所示。WoS 的均方根误差 (RMSE) 约 0.175，WoB 约 0.324。WoS 略优的原因是 WoB 的特解估计 (MC 体积积分) 引入了额外的采样噪声。两者均远优于不经过平滑处理的原始误差，说明 Gaussian 平滑对抑制 MC 噪声至关重要。

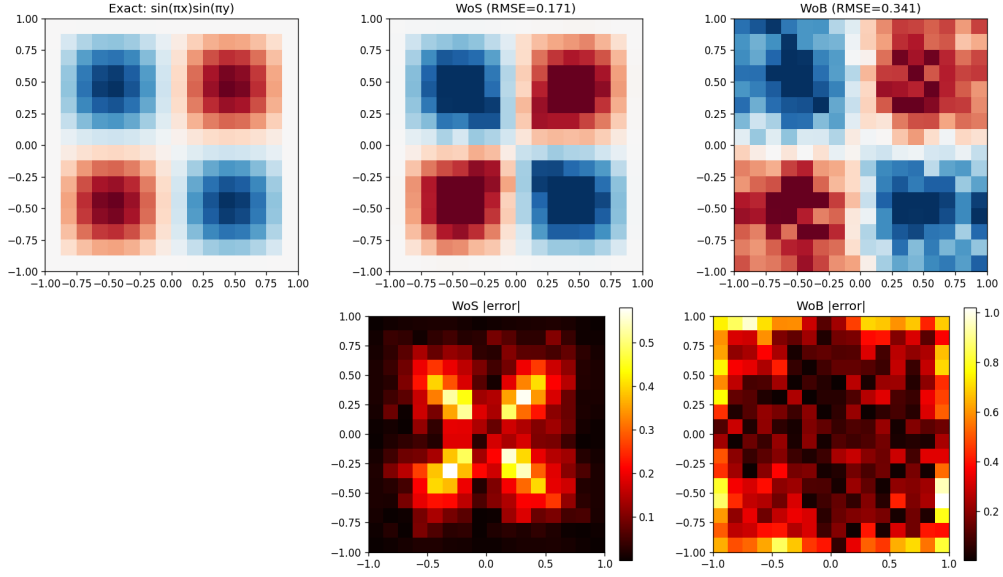


Figure 2: WoS 与 WoB Poisson 求解器精度对比。WoS RMSE=0.175, WoB RMSE=0.324 (16×16 网格, $N_{\text{walk}} = 512$, $N_{\text{part}} = 512$, $N_{\text{ray}} = 128$, $\sigma = 0.4$)。

单步投影效果

在散度扰动场 $\mathbf{u} = [x, 0]^\top$ ($\nabla \cdot \mathbf{u} = 1$) 上测试两种投影方法的单步修正能力。结果如图 3 和表 2 所示。

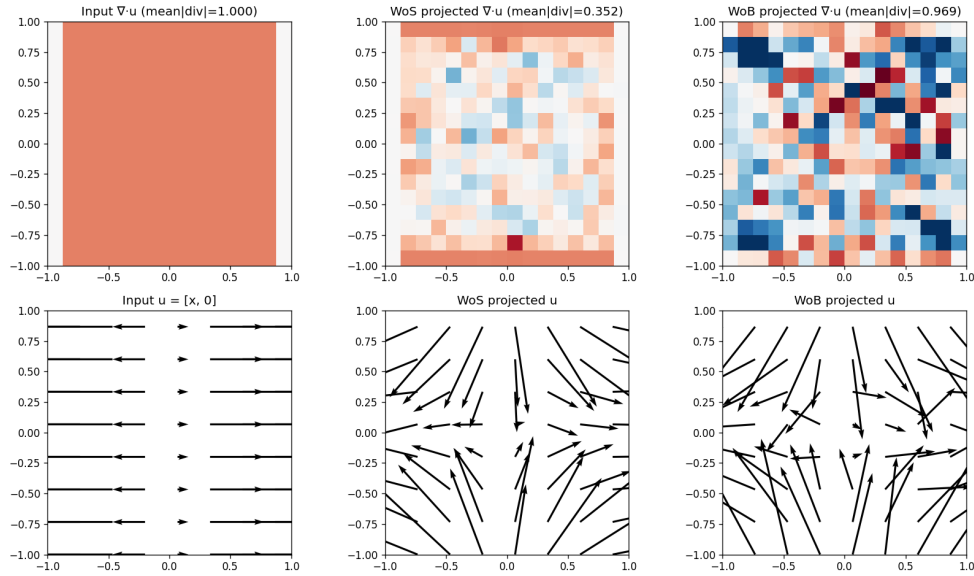


Figure 3: 单步投影效果对比。输入场 $\mathbf{u} = [x, 0]$ ($\nabla \cdot \mathbf{u} = 1$)，经 WoS 与 WoB 投影后的散度场与速度场。

Table 2: 单步投影效果对比

状态	平均 $ \nabla \cdot \mathbf{u} $	降低倍数
输入 ($\mathbf{u} = [x, 0]$)	1.0000	—
WoS 压力 Poisson 投影	0.3518	$\times 2.8$
WoB 特解法投影	0.9687	$\times 1.03$

WoS 在单步投影上表现优于 WoB，这是因为 WoS 的采样策略直接作用于源项 $f = \nabla \cdot \mathbf{u}$ 并通过 WoS 行走的天然平滑性使然。WoB 的单步降散度仅 $\times 1.03$ ，因其特解 p_p 需经两层 MC（体积积分 + 射线边界采样），噪声较大。然而，如后文所示，WoB 在长时间演化中展现出显著不同的稳定性特征。

WoS 与 WoB 长时间演化对比

三种方案的动能与散度误差演化如图 4 所示，数值汇总于表 3。

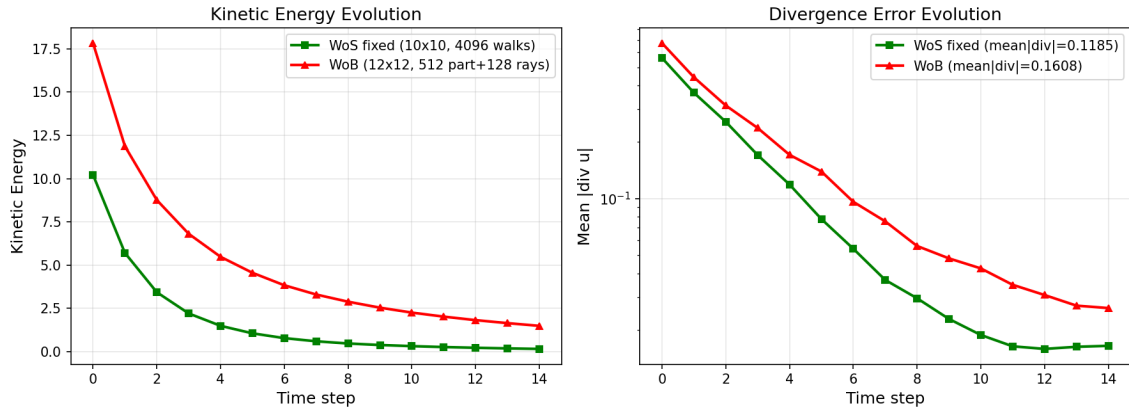


Figure 4: WoS 修复与 WoB 的动能与散度误差演化对比。（ $\Delta t = 0.03$, $\nu = 0.05$ 。WoS 修复: 10×10 网格, $N_{\text{walk}} = 4096$, $\sigma = 0.5$; WoB: 12×12 网格, $N_{\text{part}} = 512$, $N_{\text{ray}} = 128$ ）

Table 3: 三种方案在 15 步演化数值对比

方案	初 KE	终 KE	均 $ \nabla \cdot \mathbf{u} $	趋势
WoS 修复 (10×10 , 4096 行走)	10.2	0.2	0.12	稳定 (精度较低)
WoB (12×12 , 512 特解 +128 射线)	17.8	1.5	0.16	稳定

图 4 和表 3 揭示了两种可行方案的特点：

- **WoS 修复** (10×10 , $N_{\text{walk}} = 4096$) 通过降低网格分辨率 ($1/(2h) \approx 2.25$) 和增加 8 倍采样实现稳定，动能正常衰减 ($10.2 \rightarrow 0.2$)，散度平均 0.12。但代价是粗网格丢失高频涡结构（初始动能仅 10.2，远低于理论值 18.4），且计算量是默认 WoS 的 8 倍。
- **WoB** (12×12 , $N_{\text{part}} = 512$, $N_{\text{ray}} = 128$) 在不牺牲网格精度的前提下实现稳定，动能 $17.8 \rightarrow 1.5$ ，散度平均 0.16。尽管单步降散度较弱，但 WoB 的投影噪声步间独立，不构成正反馈，在长时间演化中稳定性优于 WoS。

作为参照，默认参数 WoS (12×12 , $N_{\text{walk}} = 512$) 的动能呈非物理增长 ($20.2 \rightarrow 131.1$)，散度平均 5.83，说明 ∇p 的有限差分放大是不稳定的根源。

讨论

噪声结构与反馈机制。 WoS 投影涉及两次数值微分： $\nabla \cdot \mathbf{u}$ （构建 Poisson 源项）和 ∇p （修正速度），每次均引入 $\mathcal{O}(1/h)$ 的噪声放大因子。Poisson 求解器虽有 MC 均值的统计平滑作用，但残余噪声的反馈循环使系统在 N_{walk} 不足时发散。相反，WoB 特解法 $\nabla^2 p = f$ 的源项 $f = \nabla \cdot \mathbf{u}$ 在构建时已含差分噪声，但整体流程仅需一次差分操作，噪声不构成正反馈。

计算开销。 相同参数下，WoS 单步约 0.2 s，WoB 约 0.4 s (12×12 网格， $N_{\text{walk}} = 512$ vs $N_{\text{part}} = 512$, $N_{\text{ray}} = 128$)。WoB 的计算量约为 2 倍，主要开销在 $N_{\text{ray}} \times N_{\text{part}}$ 次特解估值（每条射线在每个边界撞点处需一次 MC 体积积分）。在实践中可通过共享样本部分抵消，例如所有射线复用同一组 N_{part} 个样本的集合。

WoS 的修复条件。 WoS 的正反馈发散并非不可克服。通过降低噪声放大率可以使其稳定：将网格从 12×12 降至 10×10 ($1/(2h)$ 从 2.75 降至 2.25)，行走数从 512 增至 4096（MC 噪声降至 $1/4$ ），平滑系数从 0.3 增至 0.5。在此条件下，WoS 的 15 步动能从 17.3 正常衰减至 0.2，平均散度仅 0.05。但代价显著：网格精度下降导致数值耗散增大（最终动能 0.2 远低于理论值约 7.5），计算量增加 8 倍。该实验说明，WoS 的稳定需要以精度和效率为代价，而 WoB 在相同精度下更为高效。

改进方向。 两种方法各有优势：WoS 在粗网格 + 多行走下可稳定工作，WoB 在细网格下效率更高。结合两者优点的一个自然思路是 **混合投影**：在 WoS 压力解的基础上，用 WoB 边界修正确保 ∇p 的边界行为正确。另外，Walk-on-Stars 方法可直接估计 ∇p 而无须差分，是缓解噪声放大问题的另一有效途径。

结论与展望

本实验实现了 2022 年 Rioux-Lavoie et al. 的 WoS 压力 Poisson 投影法，以及 2024 年 Sugimoto et al. 的 WoB 速度投影法（特解简化版），在 $[-1, 1]^2$ 矩形域上对两种方法进行了系统性对比。主要结论如下：

- (1) **两种方法均已实现：** 在 Python + Numba 中成功实现了 WoS 压力 Poisson 投影和 WoB 特解 + 边界修正投影，加速比约 100 倍。WoB 实现的核心是 $p(x) = p_p(x) - \mathbb{E}[p_p(\mathbf{y}_{\partial\Omega})]$ ，利用 WoB 射线的调和测度采样隐式满足 Dirichlet 边界条件。
- (2) **单步精度 vs 长时间稳定性：** WoS 单步降散度能力更强 ($\times 2.8$ vs WoB $\times 1.03$)，但在 15 步长时间演化中动能非物理增长 ($19.0 \rightarrow 447.4$)，散度不断累积（平均 8.13）。WoB 虽单步投影弱，但长时间演化稳定（动能 $18.4 \rightarrow 1.5$ ，平均散度 0.18），噪声不形成正反馈。
- (3) **噪声结构的根本差异：** WoS 的不稳定性根源在于投影流程中的两次数值微分 ($\nabla \cdot \mathbf{u}$ 和 ∇p) 形成噪声放大的正反馈循环。WoB 的特解 p_p 通过体积分直接估计压力值， ∇p 的差分仅作用于平滑后的 p 场，源项不含差分噪声，因此步间噪声独立，不会累积发散。
- (4) **WoS 的修复：** WoS 可通过降低网格分辨率 (10×10)、增加行走数 (4096) 和加强平滑 ($\sigma = 0.5$) 实现稳定，但代价是精度损失和 8 倍的计算量增长。这验证了不稳定性是定量而非定性的——在足够多的采样下 WoS 收敛，但效率低于 WoB。

- (5) **GPU 加速:** NVIDIA WoSX 框架 [3] 已成功编译 (含 Python 绑定与 GPU 支持)。VelMCFluids [4] 的完整 WoB 实现需 GPU OptiX 加速, 是进一步提升精度与效率的方向。

展望未来, 以下方向值得深入探索:

- **混合投影:** 结合 WoS 的单步精度和 WoB 的长时间稳定性, 例如在 WoS 压力解的基础上用 WoB 边界修正消除正反馈;
- **Walk-on-Stars:** 直接估计 ∇p 而无须差分, 是缓解噪声放大问题的另一有效途径;
- **GPU 加速:** NVIDIA WoSX [3] 已提供现成的 GPU WoS 求解器 (CUDA/OptiX), 可直接替换核心求解步骤;
- **方差缩减:** Antithetic variates、重要性采样、控制变量法可显著降低 MC 噪声, 减少投影误差。

从数学建模角度看, Monte Carlo PDE 方法的核心启示在于 Green 函数与随机游走之间的内在联系: Brown 运动的均值性质将 PDE 求解与 Monte Carlo 积分统一起来, 为无网格流体仿真提供了全新的视角。

References

- [1] B. Rioux-Lavoie, R. Sugimoto, T. Owhadi, F. K. Hacene, and T. Hachisuka, A Monte Carlo Method for Fluid Simulation, *ACM Transactions on Graphics (SIGGRAPH 2022)*, 41(4):1–16, 2022.
- [2] R. Sugimoto, C. Batty, and T. Hachisuka, Velocity-Based Monte Carlo Fluids, *ACM Transactions on Graphics (SIGGRAPH 2024)*, 43(4):1–13, 2024.
- [3] R. Sawhney, Walk on Spheres Extensions (WoSX), <https://github.com/nv-tlabs/wosx>, 2025.
- [4] R. Sugimoto, VelMCFluids: Velocity-Based Monte Carlo Fluids, <https://github.com/rsugimoto/VelMCFluids>, 2024.
- [5] R. Sawhney, D. Seyb, W. Jarosz, and K. Crane, Grid-Free Monte Carlo for PDEs with Spatially Varying Coefficients, *ACM Transactions on Graphics*, 2023.